

Técnicas fundamentales de diseño de algoritmos

PROGRAMACIÓN DINÁMICA

IIC2283 – Diseño y Análisis de Algoritmos

Departamento de Ciencia de la Computación
Pontificia Universidad Católica de Chile

2025-2

Programación Dinámica

- ▶ Técnica desarrollada en los 50 por Richard Bellman para optimizar procesos de decisión de múltiples etapas
 - ▶ La palabra “programación” se refiere a “planificación”, y no a la programación de computadores.
- ▶ En este curso la usaremos como técnica de diseño de algoritmos
- ▶ La **programación dinámica** permite diseñar algoritmos que buscan de manera sistemática sobre todas las posibles soluciones de un problema dado (garantizando correctitud).

Programación Dinámica

Al igual que Dividir y Conquistar, la técnica de Programación Dinámica resuelve un problema dividiéndolo en sub-problemas más pequeños.

Ingrediente 1: Para resolver un problema usando P.D., se espera que los subproblemas estén **traslapados**:

- ▶ Un mismo subproblema es resuelto varias veces en diferentes ramas del árbol de recursión
- ▶ Eso generalmente suele conducir a tiempos de ejecución elevados (e.g., $O(2^n)$, $O(n!)$, etc.)
- ▶ Se busca resolver cada subproblema una única vez, reduciendo el tiempo a polinomial (e.g., $O(n)$, $O(n^2)$, etc.)

Programación Dinámica

Al igual que Dividir y Conquistar, la técnica de Programación Dinámica resuelve un problema dividiéndolo en sub-problemas más pequeños.

Ingrediente 2: En general, P.D. es usada para resolver problemas de optimización.

- ▶ Para que un problema de optimización pueda ser resuelto con esta técnica se debe cumplir el siguiente **principio de optimalidad para los subproblemas**:

Una solución óptima para un problema contiene soluciones óptimas para sus subproblemas

Programación Dinámica

- ▶ Si se utiliza fuerza bruta recursiva para resolver un problema de esta naturaleza:
 - ▶ un mismo **subproblema es resuelto una y otra vez** en diferentes ramas del cómputo
 - ▶ Lo cual es ineficiente!!

¿Que podemos hacer para ser más eficientes?

- ▶ Si un subproblema es resuelto por primera vez en alguna de las ramas, no debería volver a resolverse posteriormente
- ▶ Deberíamos usar la respuesta obtenida en el proceso anterior, y podar el computo en esa rama (ahorrando tiempo!!)
- ▶ **Programación Dinámica**: fuerza bruta recursiva + almacenamiento de resultados parciales

Programación dinámica: Números de Fibonacci

- ▶ Un ejemplo típico de problema con subproblemas traslapados es el cálculo del n -ésimo número en la sucesión de Fibonacci:

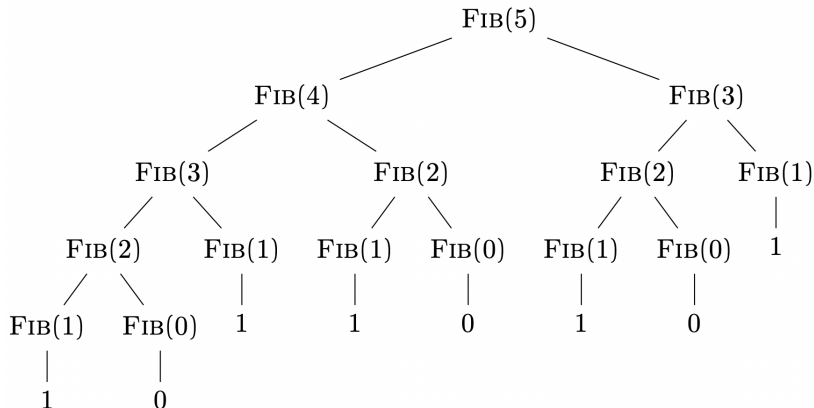
$$F(n) = \begin{cases} F(n-1) + F(n-2), & n \geq 2; \\ 0, & n = 0; \\ 1, & n = 1. \end{cases}$$

Algoritmo 6: FIB(n)

```
1 if  $n \leq 1$  then
2   | return  $n$ 
3 else
4   | return FIB( $n-1$ ) + FIB( $n-2$ )
5 end
```

Árbol de Recursión

Revisemos el árbol de recursión para $\text{FIB}(5)$:



Números de Fibonacci

- ▶ Como se puede observar, para calcular $FIB(5)$, se necesita calcular:
 - ▶ $FIB(4)$ y $FIB(3)$.
 - ▶ A su vez para resolver $FIB(4)$, necesitamos $FIB(3)$ y $FIB(2)$.
- ▶ En particular, note cómo en dos ramas distintas de la ejecución necesitamos resolver $FIB(3)$. Ilustrando el solapamiento de los subproblemas.
- ▶ Cada una de esas ramas va a repetir exactamente el mismo cómputo.
- ▶ Lo cual es ineficiente!
- ▶ Pero, ¿Qué tanto influye la resolución repetitiva de subproblemas en el tiempo de ejecución del algoritmo para Fibonacci?

Números de Fibonacci

- ▶ Note que $FIB(n) = \#$ de 1s en las hojas del árbol de recursión
 - ▶ Esos 1s son sumados a lo largo de las distintas ramas para calcular el resultado final
 - ▶ Por ejemplo, para $FIB(5) = 5$ corresponde a los 5 1s en las hojas del árbol de recursión que vimos anteriormente
- ▶ Sabemos que $FIB(n) \approx \varphi^n$,
 - ▶ en donde $\varphi = (1 + \sqrt{5})/2 = 1,618\dots$ es la conocida razón áurea
- ▶ Esto significa que el árbol de recursión de $FIB(n)$ tiene al menos φ^n hojas (aquellas con valor 1)
 - ▶ Dado que cada nodo interno tiene 2 hijos, la cantidad total de nodos en el árbol de recursión es $\Theta(\varphi^n)$
- ▶ Tenemos $\Theta(\varphi^n)$ nodos en el árbol de recursión, por lo que el tiempo de ejecución también es exponencial

Números de Fibonacci: Memoization

- ▶ A continuación, estudiamos un enfoque que **resuelve cada subproblema una única vez y luego almacena el resultado para futuros usos**
- ▶ La idea es mantener un arreglo o tabla $\text{memo}[0..n]$,
 - ▶ $\text{memo}[i]$ almacenará el resultado $\text{FIB}(i)$ cuando éste haya sido calculado
 - ▶ $\text{memo}[i] = \perp$ si $\text{FIB}(i)$ nunca ha sido calculado antes (marca especial para indicar si un subproblema ya ha sido calculado o no)
- ▶ Esta técnica se llama **memoization**

Números de Fibonacci: Memoization

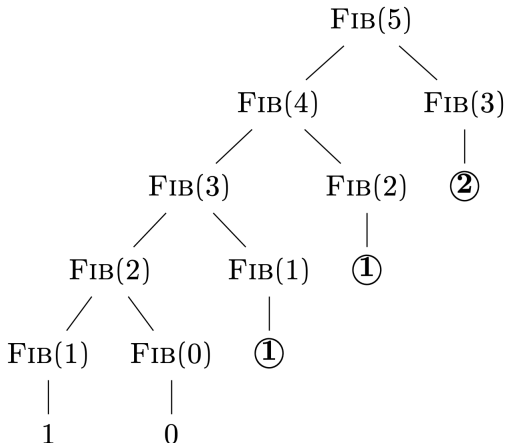
- ▶ Para Fibonacci, dado que $\text{FIB}(i) \geq 0$ para $i \geq 0$, el valor -1 se usa como marca
- ▶ Inicialmente $\text{memo}[i] = -1$ para $i = 0, \dots, n$
- ▶ A medida que avanza el computo, estas casillas se reemplazan por los resultados correspondientes

Algoritmo 20: FIB-MEMO(n)

```
1 if  $n \leq 1$  then
2   | return  $n$ 
3 else
4   | if  $\text{memo}[n] = -1$  then
5     |    $\text{memo}[n] \leftarrow \text{FIB-MEMO}(n-1) + \text{FIB-MEMO}(n-2)$ 
6   | end
7   | return  $\text{memo}[n]$ 
8 end
```

Números de Fibonacci: Memoization

- ▶ Es fácil ver que la cantidad de nodos del árbol ahora es $\Theta(n)$.
- ▶ Dado que únicamente se calcula la rama de más a la izquierda.



Números de Fibonacci: Memoization

Conclusión:

- ▶ Hemos reducido el tiempo de ejecución de exponencial a lineal!!
- ▶ A cambio, hemos aumentado el uso de espacio a $\Theta(n)$ celdas de memoria

Sin embargo, aunque use memoization, el algoritmo sigue construyendo el árbol de recursión de forma top-down

- ▶ Necesita marcas para subproblemas no calculados, e introduce ineficiencia por la recursión

Para una implementación más práctica, estudiaremos la técnica de *Programación Dinámica Bottom-up*

- ▶ Usa memoization pero construye el árbol desde las hojas a la raíz
- ▶ Permite ordenar el proceso de cómputo y llenado del memo, ya no hacen falta marcas para los subproblemas no calculados

Números de Fibonacci: P.D. Bottom-up

- ▶ Mantenemos una tabla $f[0..n]$ para almacenar los resultados de los subproblemas ya resueltos
- ▶ En particular, $f[i]$ corresponde al subproblema $FIB(i)$
- ▶ La idea es construir el árbol de cómputo de forma implícita desde las hojas hacia la raíz
 - ▶ Las hojas del árbol son los casos base de la recursión.
- ▶ En particular, los casos base son $FIB(0) = 0$ y $FIB(1) = 1$, los cuales se almacenan en la tabla:
 - ▶ $f[0] \leftarrow 0$
 - ▶ $f[1] \leftarrow 1$
- ▶ Luego, ya contamos con el material necesario para calcular $FIB(2) = FIB(1) + FIB(0) = f[1] + f[0] = 1$
- ▶ Ese valor $FIB(2) = 1$ se almacena en $f[2]$

Números de Fibonacci: P.D. Bottom-up

- ▶ El proceso continúa resolviendo problemas cada vez más grandes a partir de subproblemas que ya han sido resueltos
- ▶ Este proceso se implementa iterativamente, recorriendo el arreglo f en un orden especificado.
 - ▶ Note que para poder calcular $FIB(i)$ usando este enfoque, necesitamos haber calculado previamente $f[i - 1]$ y $f[i - 2]$
 - ▶ Eso indica que el proceso iterativo recorre el arreglo con índice $i = 2, \dots, n$, en cada iteración produciendo el resultado de $FIB(i)$ y almacenando este resultado en $f[i]$
 - ▶ De esta forma no hay necesidad de una marca que indique qué subproblemas aún no se han resuelto, como antes

Programación Dinámica Bottom-up

El algoritmo descrito anteriormente seria:

Algoritmo 21: FIB-PD(n)

```
1  $f[0] \leftarrow 0$   
2  $f[1] \leftarrow 1$   
3 for  $i \leftarrow 2, \dots, n$  do  
4   |  $f[i] \leftarrow f[i-1] + f[i-2]$   
5 end
```

- ▶ El tiempo de ejecución es $\Theta(n)$ y que ocupa $\Theta(n)$ celdas de memoria
- ▶ En ciertos casos no será necesario almacenar todo el arreglo f , en este caso solo necesitamos solo las dos posiciones casillas anteriores del arreglo.

Programación Dinámica Bottom-up

El algoritmo de a continuación calcula $\text{FIB}(n)$ en tiempo $\Theta(n)$ y utiliza espacio $\Theta(1)$:

Algoritmo 22: FIB-EC(n)

```
1 if  $n \leq 1$  then
2   | return  $n$ 
3 else
4   |  $f_2 \leftarrow 0$ 
5   |  $f_1 \leftarrow 1$ 
6   | for  $i \leftarrow 2, \dots, n-1$  do
7     |  $f \leftarrow f_1 + f_2$ 
8     |  $f_2 \leftarrow f_1$ 
9     |  $f_1 \leftarrow f$ 
10  | end
11  | return  $f_1 + f_2$ 
12 end
```

Programación Dinámica

El ejemplo anterior ilustra muchos de los elementos y principales conceptos sobre la técnica de programación dinámica. Los más importantes son:

1. Definir la ecuación de recurrencia que resuelva el problema por fuerza bruta. Es la base para construir el algoritmo de programación dinámica.
2. Definir una tabla de programación dinámica M , que se usará para la **memoization**. La tabla debe tener tantas entradas como subproblemas **distintos** existan.
3. Comenzar llenando las entradas de la tabla correspondientes a los casos base (hojas del árbol de recursión).

Programación Dinámica

4. Definir la dependencia entre entradas de M , indicando para cada entrada qué otras son necesarias para calcularla (e.g., $M[i] = M[i - 1] + M[i - 2]$).
5. Definir el orden en que las entradas de M serán llenadas (e.g., de izquierda a derecha). Esto queda claro al determinar la dependencia entre entradas.
6. Especificar cómo obtener la solución final, luego de haber llenado la tabla.

Tenga en cuenta, que en muchas situaciones no es necesario almacenar toda la tabla M a medida que se lleva a cabo el cómputo.

Midiendo la distancia entre dos strings

Muchas aplicaciones necesitan cuantificar la similitud (o disimilitud) entre dos strings:

- ▶ Biología computacional (o bioinformática)
- ▶ Correctores de ortografía
- ▶ Recuperación de información en bases de datos de documentos
- ▶ etc.

Estudiaremos una manera de medir la similitud entre dos strings que necesita programación dinámica para ser implementada

Midiendo la distancia entre dos strings

Sea Σ un alfabeto de símbolos

Sean $w_1, w_2 \in \Sigma^*$ dos strings sobre el alfabeto Σ

Vamos a utilizar distancia de Levenshtein para medir cuán similares son dos strings.

- ▶ Esta es una de las medidas de similitud entre strings más populares por lo que usualmente es llamada *edit distance*

Dados dos strings $w_1, w_2 \in \Sigma^*$, utilizamos la notación $\text{ed}(w_1, w_2)$ para la edit distance entre w_1 y w_2

- ▶ Tenemos que $\text{ed} : \Sigma^* \times \Sigma^* \rightarrow \mathbb{N}$

Tres operadores de edición sobre strings

Sea $w \in \Sigma^*$ con $w = a_1 \cdots a_n$ y $n \geq 0$

► Si $n = 0$ entonces $w = \varepsilon$

Para $i \in \{1, \dots, n\}$ y $b \in \Sigma$ tenemos que:

$$\text{eliminar}(w, i) = a_1 \cdots a_{i-1} a_{i+1} \cdots a_n$$

$$\text{agregar}(w, i, b) = a_1 \cdots a_i b a_{i+1} \cdots a_n$$

$$\text{cambiar}(w, i, b) = a_1 \cdots a_{i-1} b a_{i+1} \cdots a_n$$

Además, tenemos que $\text{agregar}(w, 0, b) = bw$

Ejemplo

<code>eliminar(hola,1)</code>	<code>=</code>	<code>ola</code>	<code>eliminar(hola,3)</code>	<code>=</code>	<code>hoa</code>
<code>agregar(hola,0,y)</code>	<code>=</code>	<code>yhola</code>	<code>agregar(hola,3,z)</code>	<code>=</code>	<code>holza</code>
<code>cambiar(hola,2,x)</code>	<code>=</code>	<code>hxla</code>			

La distancia de Levenshtein

Dados dos strings $w_1, w_2 \in \Sigma^*$, definimos $ed(w_1, w_2)$ como el **mínimo** número de operaciones eliminar, agregar y cambiar que aplicadas desde w_1 generan w_2

Ejemplo

Tenemos que $ed(\text{casa}, \text{asado}) = 3$ puesto que:

agregar(casa, 4, d) = casad

eliminar(casad, 1) = asad

agregar(asad, 4, o) = asado

La distancia de Levenshtein

Ejercicios

1. Demuestre que $\text{ed}(w_1, w_2) \leq |w_1| + |w_2|$
2. Demuestre que ed es una función de distancia, vale decir, muestre lo siguiente:
 - 2.1 $\text{ed}(w_1, w_2) = 0$ si y sólo si $w_1 = w_2$
 - 2.2 $\text{ed}(w_1, w_2) = \text{ed}(w_2, w_1)$
 - 2.3 $\text{ed}(w_1, w_2) \leq \text{ed}(w_1, w_3) + \text{ed}(w_3, w_2)$
3. Dados dos strings w_1, w_2 del mismo largo, la distancia de Hamming entre w_1 y w_2 es definida como el número de posiciones en las que tienen distintos símbolos. Denote esta distancia como $\text{hd}(w_1, w_2)$
 - 3.1 Demuestre que $\text{ed}(w_1, w_2) \leq \text{hd}(w_1, w_2)$
 - 3.2 Encuentre strings w_3 y w_4 tales que $\text{ed}(w_3, w_4) < \text{hd}(w_3, w_4)$

Calculando la distancia de Levenshtein

- ▶ Una manera de calcular $ed(w_1, w_2)$ es por fuerza bruta
 - ▶ Se deben considerar todas las posibles secuencias de operaciones de edición que permiten transformar w_1 en w_2
 - ▶ De todas esas secuencias, nos quedamos con la que necesite la menor cantidad de operaciones
 - ▶ Es importante que los strings intermedios nunca tengan longitud mayor a $|w_1| + |w_2|$, para evitar operaciones que no son estrictamente necesarias
- ▶ Luego entenderemos que esto requiere tiempo exponencial en la longitud de los strings a causa de la repetición en la solución de subproblemas
- ▶ Luego usaremos programación dinámica para implementar el mismo proceso, reduciendo el tiempo a polinomial

Notación preliminar

Dado $w \in \Sigma^*$ tal que $|w| = n$, y dado $i \in \{1, \dots, n\}$, definimos $w[i]$ como el símbolo de w en la posición i

► Tenemos que $w = w[1] \cdots w[n]$

Además, definimos el infijo de w entre las posiciones i y j como:

$$w[i, j] = \begin{cases} w[i] \cdots w[j] & 1 \leq i \leq j \leq n \\ \varepsilon & \text{en caso contrario} \end{cases}$$

La distancia y un principio optimalidad para sub-secuencias

Sean $w_1, w_2 \in \Sigma^*$ tal que $\text{ed}(w_1, w_2) = k$

Eso significa que existe una secuencia $o_1, \dots, o_k$ óptima de operaciones que aplicadas desde w_1 generan w_2 , con $k \geq 1$

Considere $0 \leq i \leq |w_1|$ y suponga que

$$o_{i_1}, o_{i_2}, \dots, o_{i_\ell}$$

(para $1 \leq i_1 \leq i_2 \leq \dots \leq i_\ell \leq k$ y $1 \leq \ell \leq k$) es la sub-secuencia de la secuencia óptima de operaciones $o_1, \dots, o_k$ que aplicadas desde $w_1[1, i]$ generan $w_2[1, j]$, con $1 \leq j \leq |w_2|$

Eso significa que la sub-secuencia debería también ser óptima, es decir,

$$\text{ed}(w_1[1, i], w_2[1, j]) = \ell$$

La distancia y un principio optimalidad para sub-secuencias

- ▶ Demostremos a continuación que eso último es cierto
- ▶ Supongamos, por contradicción, que eso no es cierto
- ▶ Eso significa que existe una secuencia de operaciones

$$o'_1, o'_2, \dots, o'_m$$

con $m < \ell$ que aplicadas desde $w_1[1, i]$ generan $w_2[1, j]$

- ▶ Así, en la secuencia original $o_1, o_2, \dots, o_k$ se podría reemplazar la sub-secuencia $o_{i_1}, o_{i_2}, \dots, o_{i_\ell}$ por la secuencia $o'_1, o'_2, \dots, o'_m$ y aún así generar w_2 desde w_1

La distancia y un principio optimalidad para sub-secuencias

- ▶ La longitud de la secuencia de operaciones resultante es

$$k - \ell + m = k - \underbrace{(\ell - m)}_{>0 \text{ porque } \ell > m} < k$$

- ▶ Eso significa $\text{ed}(w_1, w_2) < k$, lo que contradice el hecho de que $o_1, \dots, o_k$ era la secuencia óptima de operaciones
- ▶ Así, $\text{ed}(w_1, w_2)$ está compuesta se soluciones óptimas $\text{ed}(w_1[1, i], w_2[1, j])$ para $1 \leq i \leq |w_1|$ y $1 \leq j \leq |w_2|$
- ▶ Podremos resolver recursivamente el problema, sin perder optimalidad

Una definición recursiva de la distancia de Levenshtein

Fije dos strings $w_1, w_2 \in \Sigma^*$ tales que $|w_1| = m$ y $|w_2| = n$

Dados $i \in \{0, \dots, m\}$ y $j \in \{0, \dots, n\}$, definimos

$$\text{ed}(i, j) = \text{ed}(w_1[1, i], w_2[1, j])$$

Observe que $\text{ed}(w_1, w_2) = \text{ed}(m, n)$

Además, definimos el valor $\text{dif}(i, j)$ como 0 si $w_1[i] = w_2[j]$, y como 1 en caso contrario.

Una definición recursiva de la distancia de Levenshtein

Estudiamos a continuación una definición recursiva de $\text{ed}(m, n)$

Casos Base:

El algoritmo va a funcionar acortando (recursivamente) la longitud de (al menos uno de los) strings, así eventualmente alguno de ellos se hará vacío. Definimos ahora casos base que consideran eso:

1. $\text{ed}(w_1, \varepsilon) = \text{ed}(m, 0) = m$

2. $\text{ed}(\varepsilon, w_2) = \text{ed}(0, n) = n$

Así, si $i = 0$ o $j = 0$, tenemos $\text{ed}(i, j) = \max\{i, j\}$

Una definición recursiva de la distancia de Levenshtein

Casos recursivos:

Para calcular la cantidad óptima de operaciones que deben ser aplicadas desde $w_1[1]w_1[2] \cdots w_1[m]$ para obtener $w_2[1]w_2[2] \cdots w_2[n]$ hay que considerar exhaustivamente los 3 casos correspondientes a las 3 operaciones de edición que hemos definido

i) Operación Cambiar: la idea es encontrar recursivamente la secuencia óptima de operaciones para transformar $w[1, m-1]$ en $w_2[1, n-1]$ y finalmente cambiar $w_1[m]$ por $w_2[n]$

$$w_1[1, m] \underbrace{\quad \rightsquigarrow \quad}_{\text{ed}(m-1, n-1) \text{ ops}} w_2[1, n-1]w_1[m] \xrightarrow{\text{cambiar } n, w_2[n]} w_2[1, n]$$

La cantidad total de operaciones de esta alternativa es

$$\text{ed}(m-1, n-1) + \text{dif}(m, n)$$

Una definición recursiva de la distancia de Levenshtein

ii) **Operación Agregar**: la idea es encontrar recursivamente la secuencia óptima de operaciones para transformar $w_1[1, m]$ en $w_2[1, n - 1]$ y luego agregar $w_2[n]$ (que es lo que está faltando para obtener $w_2[1, n]$)

$$w_1[1, m] \underbrace{\quad \rightsquigarrow \quad}_{\text{ed}(m, n-1) \text{ ops}} w_2[1, n - 1] \xrightarrow{\text{agregar } n-1, w_2[n]} w_2[1, n]$$

La cantidad total de operaciones de esta alternativa es

$$\text{ed}(m, n - 1) + 1$$

Una definición recursiva de la distancia de Levenshtein

iii) **Operación Eliminar:** la idea es encontrar recursivamente la secuencia óptima de operaciones para transformar $w_1[1, m-1]$ en $w_2[1, n]$ y luego eliminar $w_1[m]$ (que es lo que está faltando para obtener $w_2[1, n]$)

$$w_1[1, m] \underbrace{\quad \rightsquigarrow \quad}_{\text{ed}(m-1, n) \text{ ops}} w_2[1, n] w_1[m] \xrightarrow{\text{eliminar } n+1} w_2[1, n]$$

La cantidad total de operaciones de esta alternativa es

$$\text{ed}(m-1, n) + 1$$

Una definición recursiva de la distancia de Levenshtein

La alternativa que produzca el menor número de operaciones es la que define la distancia de Levenshtein

Resumiendo, tenemos la siguiente definición recursiva para la función ed :

$$ed(m, n) = \begin{cases} \text{máx}\{m, n\} & m = 0 \text{ o } n = 0 \\ \text{mín} \begin{cases} ed(m-1, n-1) + \text{dif}(m, n) \\ ed(m, n-1) + 1, \\ ed(m-1, n) + 1, \end{cases} & m > 0 \text{ y } n > 0 \end{cases}$$

Notar cómo el algoritmo considera todas las posibles secuencias de operaciones para obtener w_2 desde w_1

Una definición recursiva de la distancia de Levenshtein

El árbol de recursión es ternario

En cada rama, al menos uno de los strings reduce su longitud en 1

- ▶ Altura del árbol es $h = \min\{m, n\}$
- ▶ Cantidad de nodos de $\Theta(3^h)$

El tiempo en cada nodo es $O(1)$, por lo que el tiempo total de ejecución es $\Theta(3^h)$

- ▶ Factible sólo para strings muy cortos
- ▶ Hay aplicaciones como biología en donde los strings pueden ser largos

Una implementación recursiva de la distancia de Levenshtein

Una implementación recursiva del esquema de fuerza bruta es la siguiente:

```
EditDistance( $w_1, i, w_2, j$ )  
  if  $i = 0$  then return  $j$   
  else if  $j = 0$  then return  $i$   
  else  
     $t := \text{EditDistance}(w_1, i - 1, w_2, j - 1)$   
     $s := \text{EditDistance}(w_1, i, w_2, j - 1)$   
     $r := \text{EditDistance}(w_1, i - 1, w_2, j)$   
    if  $w_1[i] = w_2[j]$  then  $d := 0$   
    else  $d := 1$   
    return  $\min\{t + d, s + 1, r + 1\}$ 
```

A continuación, usamos programación dinámica bottom-up para implementar este mismo algoritmo

Una evaluación bottom-up de la distancia de Levenshtein

Notar que los subproblemas recursivos a resolver son de la forma:

$$\text{ed}(i, j)$$

con $0 \leq i \leq |w_1| = m$ y $0 \leq j \leq |w_2| = n$

Eso significa que cada subproblema es identificado con dos parámetros, por lo que la tabla de programación dinámica debe ser de dimensión 2

Definimos la tabla $M[0, m][0, n]$ tal que $M[i][j] = \text{ed}(i, j)$

- ▶ Se lee: la entrada $M[i][j]$ almacenará el resultado de evaluar $\text{ed}(i, j)$

Ahora tenemos que definir cómo llenar la tabla para evitar repetir la solución de subproblemas

Una evaluación bottom-up de la distancia de Levenshtein

Primero comenzamos llenando la tabla con los casos base (las hojas del árbol de recursión)

Notar que

- ▶ $\text{ed}(i, 0) = i$, para todo $0 \leq i \leq |w_1|$
- ▶ $\text{ed}(0, j) = j$, para todo $0 \leq j \leq |w_2|$

Entonces,

- ▶ $M[i][0] = i$, para todo $0 \leq i \leq |w_1|$
- ▶ $M[0][j] = j$, para todo $0 \leq j \leq |w_2|$

Una evaluación bottom-up de la distancia de Levenshtein

Luego, llenamos las entradas $M[i][j]$ para $i > 0$ y $j > 0$

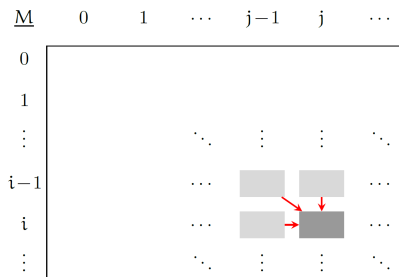
Notar que

$$\begin{aligned} M[i][j] &= \text{ed}(i, j) = \min \begin{cases} \text{ed}(i-1, j-1) + \text{dif}(i, j) \\ \text{ed}(i, j-1) + 1, \\ \text{ed}(i-1, j) + 1, \end{cases} \\ &= \min \begin{cases} M[i-1, j-1] + \text{dif}(i, j) \\ M[i, j-1] + 1, \\ M[i-1, j] + 1, \end{cases} \end{aligned}$$

Una evaluación bottom-up de la distancia de Levenshtein

Notar cómo la entrada $M[i][j]$ de la matriz depende de las entradas:

- ▶ $M[i-1][j-1]$ (operación cambiar)
- ▶ $M[i][j-1]$ (operación agregar)
- ▶ $M[i-1][j]$ (operación eliminar)



Dada esta dependencia, M debe llenarse por filas (de izq. a derecha), por columnas (de arriba a abajo) o por diagonales (comenzando en la esquina superior izq.)

Una evaluación bottom-up de la distancia de Levenshtein

Para concluir, note que

$$\text{ed}(m, n) = M[m][n]$$

por lo que la entrada $M[m][n]$ va a contener la solución al problema luego del proceso indicado

El tiempo de ejecución es $\Theta(m \cdot n)$

El espacio adicional utilizado es $\Theta(m \cdot n)$ celdas de memoria

Una evaluación bottom-up de la distancia de Levenshtein

El algoritmo resultante es el siguiente:

EditDistance-PD(w_1, w_2)

$m \leftarrow |w_1|$

$n \leftarrow |w_2|$

for $i \leftarrow 0$ **to** m **do** $M[i][0] \leftarrow i$

for $j \leftarrow 0$ **to** n **do** $M[0][j] \leftarrow j$

for $i \leftarrow 1$ **to** m **do**

for $j \leftarrow 1$ **to** n **do**

$$M[i][j] = \min \begin{cases} M[i-1, j-1] + \text{dif}(i, j) \\ M[i-1, j] + 1, \\ M[i, j-1] + 1, \end{cases}$$

return $M[m][n]$

Una evaluación bottom-up de la distancia de Levenshtein

Ejercicios

1. Ejecutar el algoritmo anterior para calcular $ed(\text{casa}, \text{asado})$. Debería obtener la siguiente tabla:

		a	s	a	d	o
	0	1	2	3	4	5
c	1	1	2	3	4	5
a	2	1	2	2	3	4
s	3	2	1	2	3	4
a	4	3	2	1	2	3

Una evaluación bottom-up de la distancia de Levenshtein

A partir de la tabla podemos obtener una secuencia óptima de operaciones para transformar casa en asado:

		a	s	a	d	o
c	0	1	2	3	4	5
a	1	1	2	3	4	5
s	2	2	1	2	3	4
a	3	3	2	1	2	3
a	4	4	3	2	1	2

Estas operaciones son las siguientes:

eliminar(casa, 1) = asa
agregar(asa, 3, d) = asad
agregar(asad, 4, o) = asado